

Google Vertex AI — Cheat sheet

HEIG-VD IST 2025/26 · Auteurs : Essinger Benoît & Rothen Evan · Plateforme ML/IA managée de Google Cloud · SDK `google-cloud-aiplatform` · réf. 2026

Ce que ça fait — Plateforme **ML/IA unifiée et managée** sur tout le cycle de vie : *data* → *entraînement* (**AutoML** *no-code* ou **Custom**) → *éval* → **serving** *online/batch* → *monitoring*, + **GenAI** (**Model Garden**, **Gemini**, RAG). **Intérêt** : d'un simple CSV à un **modèle déployé en ½ journée**, sans infra ni équipe MLOps. **Entrées** : CSV/Parquet (**GCS**), tables **BigQuery**, images/texte. **Sorties** : **endpoint REST** (temps réel) ou **batch** (→ GCS/BigQuery) + métriques & Explainable AI. **Interfaces** : SDK Python, CLI `gcloud ai`, API REST/gRPC, console, Terraform.

Concepts à connaître

Terme	Rôle
Dataset	Données managées (Tabular, Image, Text, Video) liées à GCS/BigQuery
AutoML	Entraîne un modèle sans coder l'algo — cœur de valeur PME
Custom Training	Ton code/conteneur (TF, PyTorch, sklearn) quand AutoML ne suffit pas
Model / Registry	Modèle entraîné, versionné et gouverné
Endpoint	Modèle déployé pour la prédiction online (REST, facturé 24/7)
Batch prediction	Job ponctuel de scoring de masse — pas de coût permanent
Pipelines / Feature Store	MLOps reproductible / features centralisées (avancé)
Model Garden + Gemini	Catalogue de modèles + API GenAI (pay-per-token)

Démarrer (setup, une fois)

```
gcloud auth login # s'authentifier (humain)
gcloud auth application-default login # ADC : credentials pour le SDK Python
gcloud config set project MON_PROJET # projet par défaut
gcloud config set ai/region europe-west6 # région par défaut (Zurich, data residency)
gcloud services enable aiplatform.googleapis.com storage.googleapis.com bigquery.googleapis.com
gsutil mb -l europe-west6 gs://mon-bucket # créer un bucket de staging
pip install google-cloud-aiplatform # SDK Python
```

Prérequis : projet GCP avec **facturation activée** · Python 3.9+ · rôle IAM **Vertex AI User** · crédit d'essai **300 \$ / 90 j.**

Workflow Python SDK — cas *churn* (AutoML Tabular)

```
from google.cloud import aiplatform as aip
aip.init(project="MON_PROJET", location="europe-west6", staging_bucket="gs://mon-bucket")

# 1) Dataset tabulaire depuis un CSV (ou bq://projet.dataset.table)
ds = aip.TabularDataset.create(display_name="clients_churn",
                               gcs_source="gs://mon-bucket/clients.csv")
# 2) Entraînement AutoML (cible = colonne 'churned')
job = aip.AutoMLTabularTrainingJob(display_name="churn-automl",
                                   optimization_prediction_type="classification",
                                   optimization_objective="maximize-au-prc") # classes déséquilibrées
model = job.run(dataset=ds, target_column="churned",
                budget_milli_node_hours=1000) # 1 node-heure (~21 $)
# 3) Évaluation
for ev in model.list_model_evaluations(): print(dict(ev.metrics).get("auPrc"))
# 4a) Prédiction BATCH (recommandé hors temps réel → bien moins cher)
model.batch_predict(job_display_name="churn-batch", instances_format="csv",
                   gcs_source="gs://mon-bucket/a_scorer.csv",
                   gcs_destination_prefix="gs://mon-bucket/predictions/")
# 4b) Prédiction ONLINE (temps réel) – PENSE À UNDEPLOY ENSUITE
endpoint = model.deploy(machine_type="n1-standard-4")
endpoint.predict(instances=[{"anciennete_mois": "3", "type_contrat": "mensuel",
                             "facture_mensuelle": "99.9", "nb_tickets_support": "5"}])
endpoint.undeploy_all(); endpoint.delete() # ● stoppe la facturation
```

Datasets

Commande	Effet
<code>gcloud ai datasets list</code>	lister
<code>gcloud ai datasets create --display-name=N ...</code>	créer un dataset
<code>gcloud ai datasets describe ID</code>	détails
<code>gcloud ai datasets delete ID</code>	supprimer

Modèles & registry

Commande	Effet
<code>gcloud ai models list</code>	lister les modèles
<code>gcloud ai models describe ID</code>	détails / versions
<code>gcloud ai models upload --display-name=N ...</code>	importer un modèle custom
<code>gcloud ai models delete ID</code>	supprimer

Prédiction batch & jobs

Commande	Effet
<code>gcloud ai batch-prediction-jobs create --model=ID ...</code>	scoring batch
<code>gcloud ai batch-prediction-jobs list</code>	suivre les jobs batch
<code>gcloud ai custom-jobs create --worker-pool-spec=...</code>	entraînement custom
<code>gcloud ai custom-jobs stream-logs JOB</code>	logs d'un job en direct

Endpoints (serving online)

Commande	Effet
<code>gcloud ai endpoints list</code>	lister les endpoints
<code>gcloud ai endpoints create --display-name=N</code>	créer un endpoint
<code>gcloud ai endpoints deploy-model EP --model=ID --machine-type=...</code>	déployer un modèle
<code>gcloud ai endpoints predict EP -json-request=req.json</code>	prédire (temps réel)
<code>gcloud ai endpoints undeploy-model EP --deployed-model-id=ID</code>	● stoppe la facture
<code>gcloud ai endpoints delete EP</code>	supprimer l'endpoint

Opérations & infos

Commande	Effet
<code>gcloud ai operations list</code>	opérations en cours
<code>gcloud ai operations describe OP</code>	suivre une opération longue
<code>gcloud ai endpoints describe EP</code>	modèles déployés sur un endpoint

GenAI / Gemini (Model Garden)

```
import vertexai
from vertexai.generative_models import GenerativeModel
vertexai.init(project="MON_PROJET", location="europe-west6")
model = GenerativeModel("gemini-2.5-flash") # ou gemini-2.5-pro
print(model.generate_content("Explique le churn en une phrase.").text)
# RAG / recherche sémantique : TextEmbeddingModel.from_pretrained("text-embedding-005")
```

GenAI = **pay-per-token** : Gemini 2.5 Flash ≈ 0,30 \$/1M in · Pro ≈ 1,25 \$/1M in. Pas d'infra à gérer, pas d'always-on.

Maîtriser les coûts (facturation à l'usage, à la seconde — pas d'abonnement)

Poste	Ordre de grandeur
Entraînement AutoML	≈ 21 \$/node-heure
Endpoint online n1-standard-4	≈ 0,22 \$/h → ~160 \$/mois 24/7
Prédiction batch	compute du job seulement
Gemini (token)	Flash ≈ 0,30 \$/1M in

```
gcloud ai endpoints list --region=europe-west6 # traquer les endpoints always-on
gcloud billing budgets create --billing-account=ACCT \
--display-name="vertex" --budget-amount=100USD # budget + alerte
```

💡 **Scénario PME (churn, ~50k clients, scoring mensuel)** : endpoint online 24/7 ≈ **217 \$/mois (~CHF 195)** · en batch ≈ **65 \$/mois (~CHF 60)**. Batch vs online divise la facture par ~3 ; un endpoint oublié est le piège n°1 → undeploy systématique.

Bonnes pratiques & quand l'utiliser

Bonnes pratiques

- **Batch par défaut** ; online seulement si temps réel requis.
- **undeploy / delete** les endpoints + **budgets/alertes**.
- **Région UE/CH** (europe-west6) pour la résidence des données.
- **Formats ouverts** (Parquet/SQL) → limite le **vendor lock-in**.

Utiliser quand / éviter

- **Utiliser** : déjà sur GCP/BigQuery, pas d'équipe MLOps, besoin d'aller vite, données tabulaires/images.
- **Éviter** : besoin simple & stable (scikit + cron), trafic faible, exigence forte de portabilité.